Larkfield Parcels: building a relational database for a regional courier

Uday Kiran Penjarla | Student ID: 25044896 | November 2026 | **Notebook:**
https://github.com/udaypenjarla/Dataminingproject.git

1. What Larkfield needs from its data

This database belongs to the fictional courier Larkfield Parcels, which has eight depots in the East of England region and neighbouring countries. It applies to the depots, the vans and drivers, the business customers that send parcels, every parcel booked between March and September 2026, and each attempt to deliver the parcel. It could be used by a depot manager to respond to daily queries. How many parcels are delivered on time? Who are the top customers? Will electric vans have a reduced range in cold weather? This file contains 48,597 lines in 8 tables. These are delivery_attempts (18,237), consignments (16,096) and driver_shifts (13,349) the largest of these).

2. Making the parcel records

No data were purchased or downloaded. All of it is done by one Python notebook. The random numbers are drawn with NumPy and the driver name and company name are set with Faker en_GB (Faker, 2026). The seed is 9274 and the same database is created by the notebook each time it is run. The rules are mostly applied across the entire table at once and most columns are drawn for the table, e.g. electric vans cannot be lorries, using np.where.

I have attempted to make the numbers react as if it were really a courier. The shape of the parcel weight is log-normal; i.e. most parcels are relatively light (median is 1.71 kg) and a few are approaching the 30 kg limit. The weight increases the length of the box. The day's busiest times for bookings are around midday as online shops print labels. The prices are based on a matrix of four services and five weight bands, with larger accounts receiving a discount of up to 18%. No vans are in two places at once as drivers only drive when they have a shift and using the van provided that day. Real UK age codes like 24 (spring 2024) or 74 (autumn 2024) are used on the van number plates. Odometer readings carry on from shift to shift. 87.0% of the first attempts are successful, parcels are ressented up to three times in case of failure and 231 parcels have already been returned to the sender. This is a well-documented phenomenon: cold weather reduces electric van ranges, resulting in fewer kilometres on cold days. There are a few parcels (35) that are lost and have no attempts.

Real courier data has gaps and repeats, so a later notebook step breaks some rows deliberately. This is what it does:

Problem	Where	Rows	Real-life cause
Missing	senders.phone	28	Account opened online with no phone number
Missing	consignments.declared_value_gbp	803	Value is optional when booking
Missing	consignments.recipient_phone	565	Shop did not pass on the number
Missing	driver_shifts.end_odometer_km	200	Driver forgot to log the reading
Duplicate	senders	14	Business opened a second account
Duplicate	consignments	96	Label printed twice for the same parcel

These numbers were counted in the final database. Some are a little above the notebook's printout, because a duplicated record can carry over a gap that was already there.

3. Eight tables for depots, vans and parcels

The same fact can only be stored in one table. This is the essence of Codd's (1970) relational model, and this means that the town a depot is in is a depot, nothing more. There are 11 foreign keys between the tables. The notebook enables foreign key enforcement before loading anything and SQLite won't allow a row if its parent is missing. None of the columns remain unchecked; every column has either a data type AND NOT NULL, a CHECK, a key.

depots (8 rows). Most other tables reference this table so details of a depot are in a single location. The code should be UNIQUE and consist of three capital letters; the region should be one of a short list of regions.

service_rates (20 rows). This is the price list, 1 line for each service, each weight band. Those two columns are together a primary key and all prices must be greater than 0.

vehicles (79 rows). Vans are assigned to a depot. The plate should be the same as in the UK and the composite UNIQUE id on depot_id and fleet_no prevents two vans sharing the same fleet number in the same depot.

drivers (94 rows). Staff changes at their own schedule, so they have their own table. Each driver is associated with a foreign key depot and a contract type from a list of available contracts.

senders (714 rows). Business customers have their parcels separated from them. There are four levels of accounts, emails and landline checked against a pattern.

consignments (16,096 rows). One row per parcel. The tracking number is UNIQUE, weight is limited to 30 kg, and each parcel has a composite foreign key that is related to a real row in service_rates.

driver_shifts (13,349 rows). This will allow you to see who drove which van and on which day. Driver and Date is the key, and the ending Value on Odometer must be greater than or equal to the starting Value on Odometer.

delivery_attempts (18,237 rows). Sometimes a parcel may require more than one attempt, therefore an attempt has its own table, keyed on consignment_id and attempt_no (which may be 1, 2 or 3).

There were some rules that required some thinking. A composite primary key is used in the price grid and so consignment is related to it through a two-column foreign key (as illustrated in figure 1 in the next section). This prevents a parcel being booked on a service that isn't available and on a weight band that doesn't exist. The key of the shift table is: driver,date, because a driver may only have one shift per day. A van can never go backwards as the end odometer reading is compared to the initial reading. Did NOT make business_name UNIQUE because I wanted duplicates of accounts to use for testing.

All four levels of measurement from Stevens (1946) are used:

Level	Columns	Reason
Nominal	fuel_type, contract_type, outcome, region	Categories with no order
Ordinal	service_level, weight_band, account_tier, vehicle_class	Ordered, but gaps are not equal
Interval	ambient_temp_c, booked_at, shift_date, opened_year	Equal steps, no true zero
Ratio	weight_kg, price_gbp, distance_km, payload_kg	True zero, so twice as much makes sense

4. Diagram of the courier tables

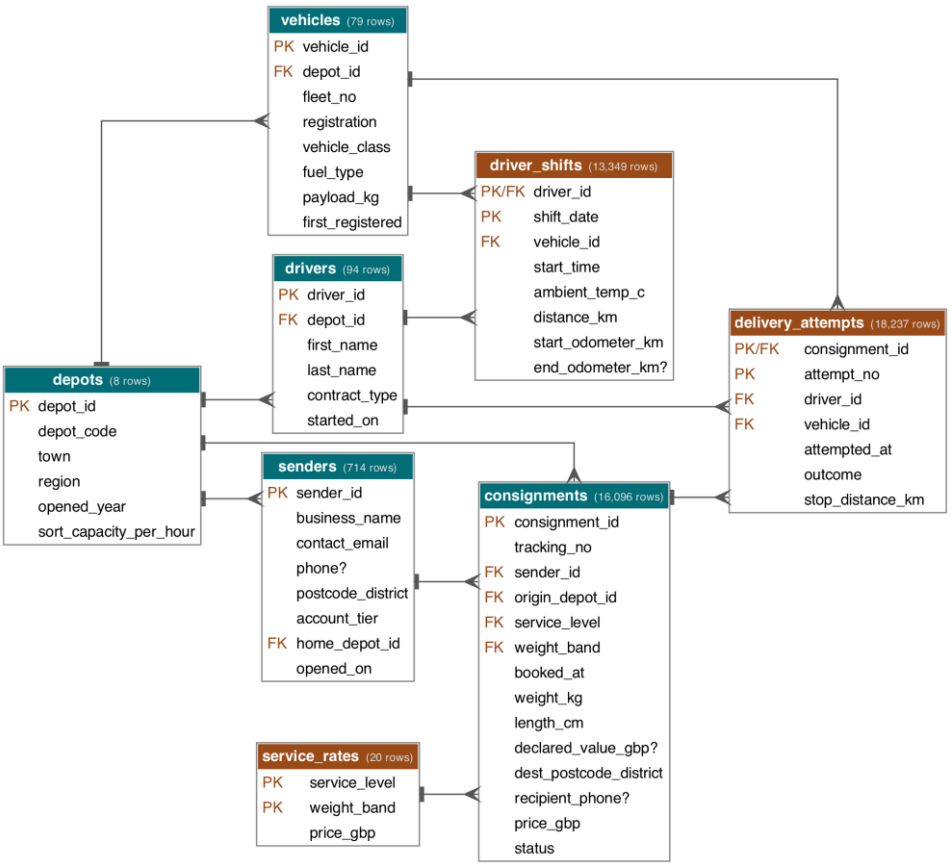


Figure 1. Larkfield tables and links. Brown tables have composite primary keys and "?" marks a column that may be empty.

5. Driver and customer privacy

The names of the people and companies are fictional; therefore no real person's information has been used. This allows the system to be tested in a safe environment, without any actual customer data involved. However, a made up driver or firm can have the same name as a real one, so any copy of the data should clearly indicate that the data is synthetic.

If it was a live courier, it would have to comply with the Data Protection Act 2018 as driver names, sender contact information and recipient phone numbers could all relate to a living person. I just added the information I needed to post a parcel: destination stored as postcode district, no name of the person who was the recipient stored. Recipient phone numbers should be deleted shortly after delivery as they are only required on the day of delivery. Delivery records provide ID of delivery drivers only. That doesn't mean they're anonymous, though, as they get joined to the

drivers table and the name is then joined back. Access to tables of shift and attempt times might be particularly important to track a driver closely; therefore access should be carefully controlled and drivers should be informed about the uses of the data.

6. Six checks on the finished parcel file

I ran six checks on the finished file. Each one is a short piece of SQL, and the table gives the answer it returned. The duplicate count matches the number of double-printed labels added in the notebook.

Question	SQL I ran	Answer
Does SQLite see any damage?	PRAGMA integrity_check	"ok"
Rows pointing at a missing parent	PRAGMA foreign_key_check	0 found
Rows in the biggest table	SELECT COUNT(*) FROM delivery_attempts	18,237
Double-printed labels	GROUP BY sender, time, weight, district HAVING COUNT(*) > 1	96 labels
Senders with no phone	SUM(phone IS NULL) on senders	28 senders
Delivered at first attempt	share of good outcomes WHERE attempt_no = 1	87.0%

Reference list

Codd, E.F. (1970) 'A relational model of data for large shared data banks', *Communications of the ACM*, 13(6), pp. 377–387.
Data Protection Act 2018, c. 12. Available at: https://www.legislation.gov.uk/ukpga/2018/12/contents (Accessed: 24 October 2026).
Faker (2026) *Faker documentation, version 40.43.0*. Available at: https://faker.readthedocs.io/ (Accessed: 19 October 2026).
Stevens, S.S. (1946) 'On the theory of scales of measurement', *Science*, 103(2684), pp. 677–680.